LangChain Modules

Generative AI

Module 3 – Lesson 1

Today's activities

- Review of module 2
- Introduction to LangChain
- LangChain modules
- Memory

Review: Core dimensions of responsible AI

Controllability

Having mechanisms to monitor and steer AI system behavior

Privacy & Security

Appropriately obtaining, using and protecting data and models

Safety

Preventing harmful system output and misuse

Fairness

Considering impacts on different groups of stakeholders

Veracity & Robustness

Achieving correct system outputs, even with unexpected or adversarial inputs

Explainability

Understanding and evaluating outputs generated by an AI system

Transparency

Enabling stakeholders to make informed choices about their engagement with an AI system

Governance

Incorporating best practices into the AI supply chain, including providers and deployers

LangChain

LangChain

- One of the most popular frameworks for LLM-based applications
- A framework for developing applications powered by language models
  - Modular components
  - Flexible and scalable
  - Long list of third-party integrations

LangChain Use Cases

- Text generation
- Summarization
- Conversational chatbots
- Code implementation/understanding
- Retrieval augmented generation (RAG)
- Reasoning agents

- **And so much more!**

Why LangChain? Prompt, don’t train!

- Focus on **using** existing LLMs instead of training or fine-tuning
- Several LLMs are generally very capable
  - Can be adapted using prompts and scripts
- Training or fine-tuning is usually more expensive and demanding
- Compatible with existing APIs

Why LangChain? Standard interface

- Common interfaces for components
  - Growing number of models, APIs, services, etc.
  - Diverse formats and structures to use them

- Easily switch between providers
  - Modular framework

Why LangChain? Orchestration

- AI applications are becoming more complex!

- Efficiently connect elements into custom controlled flows
  - Loop over the task plan until goal achieved
  - Persist data across multiple stages
  - Include humans in decision making, reviewing, etc.

- Accomplish diverse tasks

Why LangChain? Observability and evaluation

- **Traceability** features to observe outputs at all stages

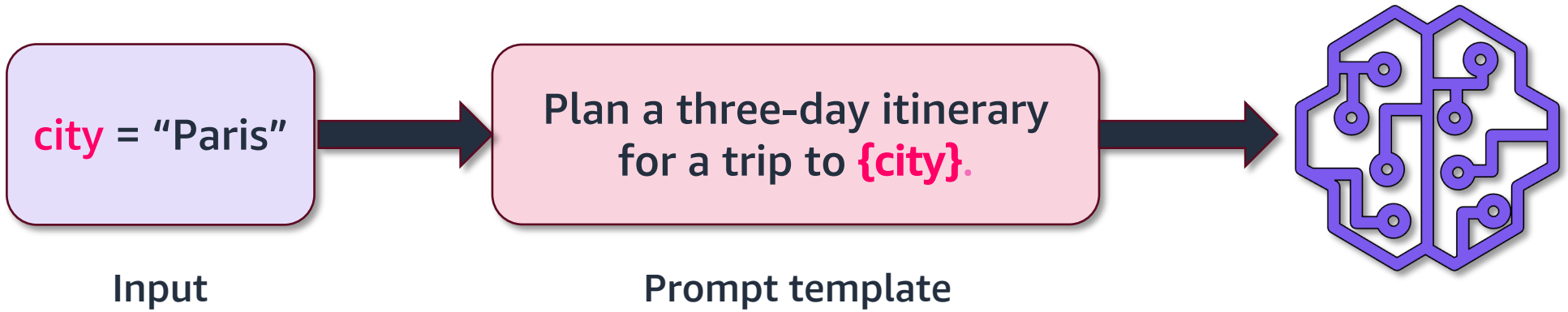
- Test portions of the application without running the whole workflow

- Tools in measuring and achieving desired outcomes
  - Latency
  - Performance
  - Cost

LangChain Modules

Prompt templates

- Parameterized inputs serving as pre-defined recipes for LLMs
  - Templates may include instructions, context, few-shot examples etc
- Model agnostic
- Reusable



Output parsers

- Parse the output of LLM into various formats
  - **StrOutputParser**
  - **JsonOutputParser**
  - **XMLOutputParser**
  - **BooleanOutputParser**

- Useful of the output of the LLM needs to adhere to specific formats for downstream tasks

Chains

- Enables building complex applications by **chaining** modules
  - **Sequence of calls** to LLM, tool, etc.
- **Simplifies** complex tasks using multiple steps and components
- Easy to **switch components** without disrupting the entire workflow
- Easiest to use LCEL
  - **LangChain Expression Language**

LangChain Expression Language (LCEL)

- Easily compose chains

- **Several benefits**
  - Streaming support
  - Parallel execution
  - Configure retried and fallbacks
  - Access intermediate steps
  - Specify input/output schema for conformity

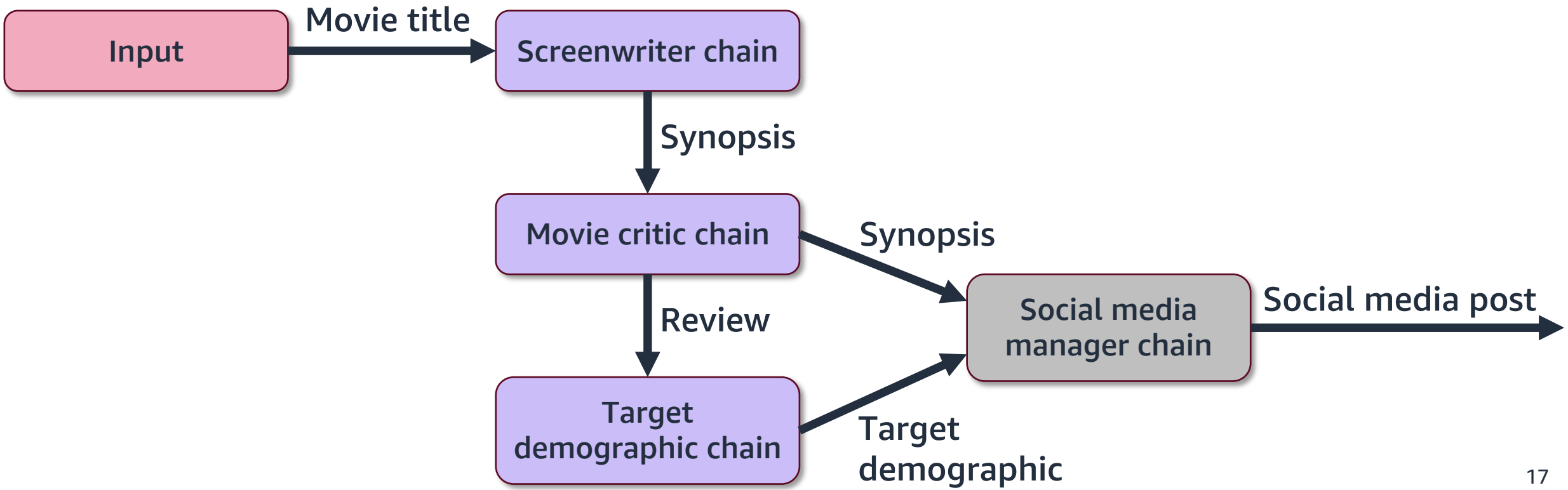
Simple LLM chain

- Combine an LLM, prompt template and output parser

```
# Define prompt template
prompt_template = PromptTemplate.from_template(
        "Tell me a {content_type} about {topic}."
)

# Create a simple LLM Chain
chain = prompt_template | llm | StrOutputParser()

chain.invoke({content_type:"bedtime story", topic:"a cat"})

>> Once upon a time, there was a little black cat named Midnight.
Midnight was a very curious cat, and …
```

Connecting multiple chains

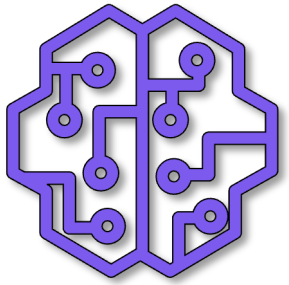
- Connect multiple chains together to create complex workflows
  - Define the flow using the proper input and output variables



Memory

Simple prompting

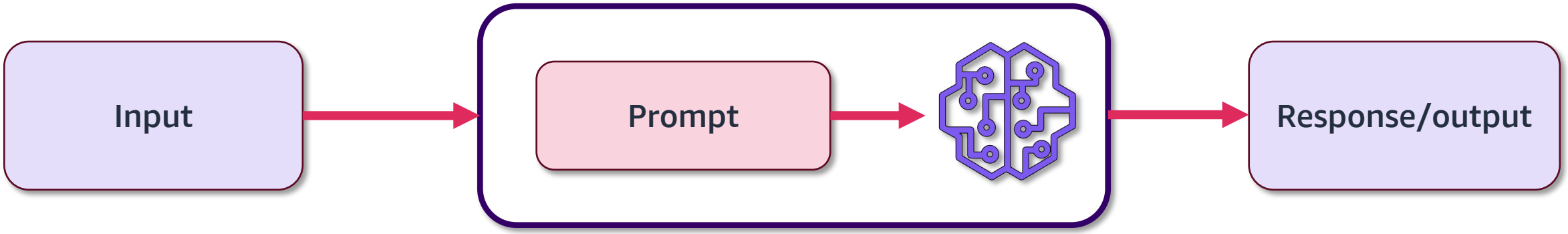
What seems to be the problem?



Prompt 1	Hi, my name is Andy	Hi Andy, it is nice to meet you.
Prompt 2	What is my name?	I don't know. What do I call you?

Simple LLM application

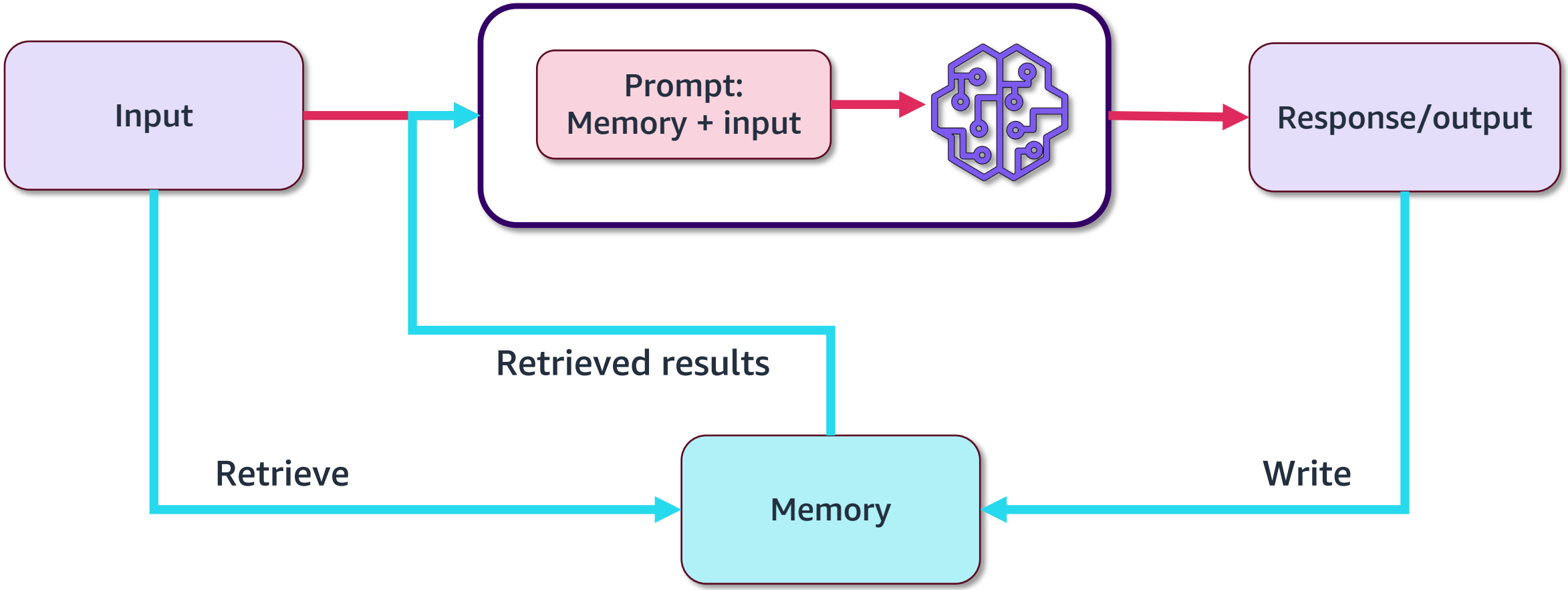
- Without any component persisting context, the LLM is unaware of prior interactions
  - » **LLMs are stateless**



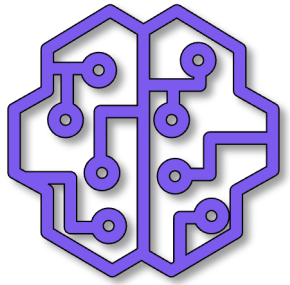
Memory

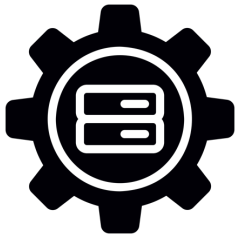
- Maintaining information that the LLM can refer to
  - Remembering **past interactions**
  - Persisting **context, roles, rules**, and so on
- Allows the LLM to dynamically store and access information
- Commonly used for chatbots and other conversational applications

LLM Application with memory



Prompting with memory





Memory

Hi, my name is Andy	Hi Andy, it is nice to meet you.	Human: Hi, my name is Andy. AI: Hi Andy, it is nice to meet you.
What is my name?	You introduced yourself as Andy.	Human: Hi, my name is Andy. AI: Hi Andy, it is nice to meet you. Human: What is my name? AI: You introduced yourself as Andy

Conversational Memory

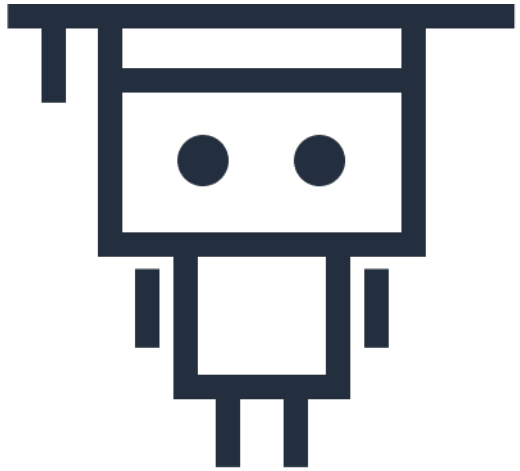
- Storing all interactions and retrieving them as history
  - **Perfect recall**

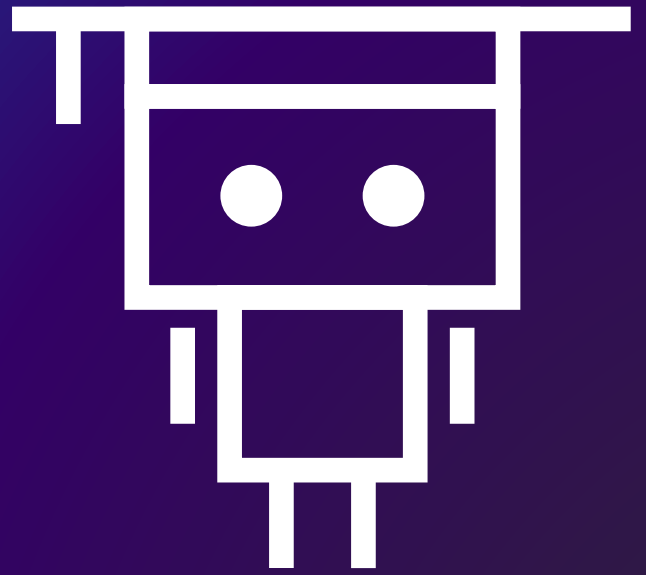
```
# Create a ConversationBufferMemory
memory = ConversationBufferMemory()

# Save context into memory
memory.save_context({"input":"Hi"},{"output":"What's up?"})

# Output context from memory
memory.load_memory_variables({})

>> {'history': 'Human:Hi\nAI: What's up?'}
```

- What if the conversation is too long?
  - **We will address this in the next lesson!**

Next lesson

- This lesson covered LangChain and a few modules offered by the framework.
- In the next lesson, you will learn about developing conversational applications.





Thank you!